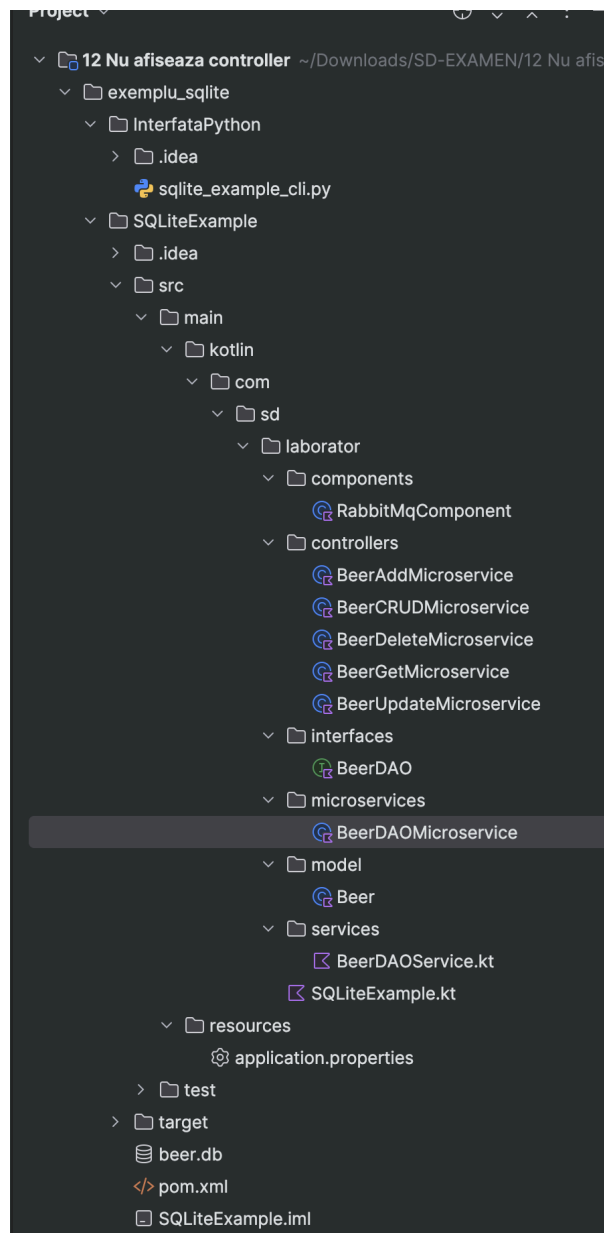


Pornind de la exemplul 1 din laboratorul 6 să se modifice acesta astfel încât să fie câte un microserviciu separat pentru fiecare operație CRUD, iar acestea să fie accesate prin intermediul unui model de proiectare de tip poartă API în interiorul altui microserviciu.

Dacă se consideră necesar se mai pot crea oricâte servicii se dorește conform gândirii studentului.

Se va utiliza orchestrarea și respectarea principiilor SOLID pentru microservicii. La prezentare se va arăta diagrama de clase și se va explica aceasta împreună cu gradul de respectare a principiilor. Apoi va urma discuția pe cod.



RabbitMqComponent

```
package com.sd.laborator.components

import org.springframework.amqp.rabbit.connection.CachingConnectionFactory
import org.springframework.amqp.rabbit.connection.ConnectionFactory
import org.springframework.amqp.rabbit.core.RabbitTemplate
import org.springframework.beans.factory.annotation.Value
import org.springframework.context.annotation.Bean
import org.springframework.stereotype.Component

@Component
class RabbitMqComponent {
    @Value("\${spring.rabbitmq.host}")
    private lateinit var host: String
    @Value("\${spring.rabbitmq.port}")
    private val port: Int = 0
    @Value("\${spring.rabbitmq.username}")
    private lateinit var username: String
    @Value("\${spring.rabbitmq.password}")
    private lateinit var password: String
    @Value("\${sqliteexample.rabbitmq.exchange}")
    private lateinit var exchange: String
    @Value("\${sqliteexample.rabbitmq.routingkey}")
    private lateinit var routingKey: String

    fun getExchange(): String = this.exchange

    fun getRoutingKey(): String = this.routingKey

    @Bean
    private fun connectionFactory(): ConnectionFactory {
        val connectionFactory = CachingConnectionFactory()
        connectionFactory.host = this.host
        connectionFactory.username = this.username
        connectionFactory.setPassword(this.password)
        connectionFactory.port = this.port
        return connectionFactory
    }

    @Bean
    fun rabbitTemplate(): RabbitTemplate = RabbitTemplate(connectionFactory())
}
```

BeerAddMicroService

```
package com.sd.laborator.controllers

import com.sd.laborator.interfaces.BeerDAO
```

```
import com.sd.laborator.model.Beer
import org.springframework.beans.factory.annotation.Autowired

class BeerAddMicroservice {
}
```

BeerCRUDMicroservice

```
package com.sd.laborator.controllers

import org.springframework.stereotype.Controller
import org.springframework.web.bind.annotation.PathVariable
import org.springframework.web.bind.annotation.RequestMapping
import org.springframework.web.bind.annotation.RequestMethod
import org.springframework.web.bind.annotation.ResponseBody

@Controller
class BeerCRUDMicroservice {
    @RequestMapping("/getbeerbyname/{name}", method = [RequestMethod.GET])
    @ResponseBody
    fun getBeerByName(@PathVariable name: String): String {
        return
khttp.get("http://localhost:8080/getbeerbyname/microservice/$name").text
    }

    @RequestMapping("/getbeerbyprice/{price}", method = [RequestMethod.GET])
    @ResponseBody
    fun getBeerByPrice(@PathVariable price: Float): String {
        return
khttp.get("http://localhost:8080/getbeerbypricemicroservice/$price").text
    }

    @RequestMapping("/getbeers", method = [RequestMethod.GET])
    @ResponseBody
    fun getBeers(): String {
        return khttp.get("http://localhost:8080/getbeersmicroservice").text
    }
}
```

BeerDeleteMicroservice

```
package com.sd.laborator.controllers

class BeerDeleteMicroservice {
}
```

BeerGetMicroservice

```
package com.sd.laborator.controllers

import com.sd.laborator.interfaces.BeerDAO
import org.springframework.beans.factory.annotation.Autowired
import org.springframework.stereotype.Controller
import org.springframework.web.bind.annotation.PathVariable
import org.springframework.web.bind.annotation.RequestMapping
import org.springframework.web.bind.annotation.RequestMethod
import org.springframework.web.bind.annotation.ResponseBody

@Controller
class BeerGetMicroservice {
    @Autowired
    private lateinit var beerDAO: BeerDAO

    @RequestMapping("/getbeerbynamemicroservice/{name}", method =
[RequestMethod.GET])
    @ResponseBody
    fun getBeerByName(@PathVariable name: String): String {
        return beerDAO.getBeerByName(name)!!
    }

    @RequestMapping("/getbeerbypricemicroservice/{price}", method =
[RequestMethod.GET])
    @ResponseBody
    fun getBeerByPrice(@PathVariable price: Float): String {
        return beerDAO.getBeerByPrice(price)!!
    }

    @RequestMapping("/getbeersmicroservice", method = [RequestMethod.GET])
    @ResponseBody
    fun getBeers(): String {
        return beerDAO.getBeers()
    }
}
```

BeerUpdateMicroservice

```
package com.sd.laborator.controllers

class BeerUpdateMicroservice {
}
```

BeerDAO

```

package com.sd.laborator.interfaces

import com.sd.laborator.model.Beer

interface BeerDAO {
    // Create
    fun createBeerTable()
    fun addBeer(beer: Beer)

    // Retrieve
    fun getBeers(): String
    fun getBeerByName(name: String): String?
    fun getBeerByPrice(price: Float): String?

    // Update
    fun updateBeer(beer: Beer)

    // Delete
    fun deleteBeer(name: String)
}

```

BeerDAOMicroService

```

package com.sd.laborator.microservices

import com.sd.laborator.components.RabbitMqComponent
import com.sd.laborator.interfaces.BeerDAO
import com.sd.laborator.model.Beer
import org.springframework.amqp.core.AmqpTemplate
import org.springframework.amqp.rabbit.annotation.RabbitListener
import org.springframework.beans.factory.annotation.Autowired
import org.springframework.stereotype.Component

@Component
class BeerDAOMicroservice {
    @Autowired
    private lateinit var beerDAO: BeerDAO

    @Autowired
    private lateinit var rabbitMqComponent: RabbitMqComponent

    private lateinit var amqpTemplate: AmqpTemplate

    @Autowired
    fun initTemplate() {
        this.amqpTemplate = rabbitMqComponent.rabbitTemplate()
    }
}

```

```

//citesc din queue1
// scriu in queue
@RabbitListener(queues = ["\${sqliteexample.rabbitmq.queue}"])
fun recieveMessage(msg: String) {
    val processed_msg = (msg.split(",").map { it.toInt().toChar()
}).joinToString(separator="")
    val (operation, parameters) = processed_msg.split('~')
    var beer: Beer? = null
    var price: Float? = null
    var name: String? = null

    // id=1;name=Corona;price=3.6
    if("id=" in parameters) {
        println(parameters)
        val params: List<String> = parameters.split(';')
        try {
            beer = Beer(
                params[0].split('=')[1].toInt(),
                params[1].split('=')[1],
                params[2].split('=')[1].toFloat()
            )
        } catch (e: Exception) {
            print("Error parsing the parameters: ")
            println(params)
            return
        }
    } else if ("price=" in parameters) {
        price = parameters.split('=')[1].toFloat()
    } else if ("name=" in parameters) {
        name = parameters.split("=")[1]
    }
    //println(parameters)
    //println(name)
    //println(price)
    //println(beer)

    try {
        var result: Any? = when (operation) {
            "createBeerTable" -> beerDAO.createBeerTable()
            "addBeer" -> beerDAO.addBeer(beer!!)
            "getBeers" -> khttp.get("http://localhost:8080/getbeers").text
            "getBeerByName" ->
khttp.get("http://localhost:8080/getbeerbyname/$name").text
            "getBeerByPrice" ->
khttp.get("http://localhost:8080/getbeerbyprice/$price").text
            "updateBeer" -> beerDAO.updateBeer(beer!!)
            "deleteBeer" -> beerDAO.deleteBeer(name!!)
            else -> null
        }
    }

```

```

        println("result: ")
        println(result)
        if (result != null) sendMessage(result.toString())
    }
    catch (e: Exception) { println("Eroare la get: $e")}
}

fun sendMessage(msg: String) {
    println("message: ")
    println(msg)
    this.amqpTemplate.convertAndSend(rabbitMqComponent.getExchange(),
rabbitMqComponent.getRoutingKey(), msg)
}
}

```

Beer

```

package com.sd.laborator.model

class Beer(private var id: Int, private var name: String, private var price:
Float) {

    var beerID: Int
        get() {
            return id
        }
        set(value) {
            id = value
        }
    var beerName: String
        get() {
            return name
        }
        set(value) {
            name = value
        }
    var beerPrice: Float
        get() {
            return price
        }
        set(value) {
            price = value
        }

    override fun toString(): String {
        return "Beer [id=$beerID, name=$beerName, price=$beerPrice]"
    }
}

```

```
}
```

BeerDAOService

```
package com.sd.laborator.services

import com.sd.laborator.interfaces.BeerDAO
import com.sd.laborator.model.Beer
import org.springframework.beans.factory.annotation.Autowired
import org.springframework.jdbc.core.JdbcTemplate
import org.springframework.jdbc.core.RowMapper
import org.springframework.stereotype.Service
import java.sql.ResultSet
import java.sql.SQLException
import java.util.regex.Pattern

class BeerRowMapper : RowMapper<Beer?> {
    @Throws(SQLException::class)
    override fun mapRow(rs: ResultSet, rowNum: Int): Beer {
        return Beer(rs.getInt("id"), rs.getString("name"), rs.getFloat("price"))
    }
}

@Service
class BeerDAOService: BeerDAO {
    //Spring Boot will automatically wire this object using
    application.properties:
    @Autowired
    private lateinit var jdbcTemplate: JdbcTemplate
    var pattern: Pattern = Pattern.compile("\\W")

    override fun createBeerTable() {
        jdbcTemplate.execute("""CREATE TABLE IF NOT EXISTS beers(
                                id INTEGER PRIMARY KEY AUTOINCREMENT,
                                name VARCHAR(100) UNIQUE,
                                price FLOAT)""")
    }

    override fun addBeer(beer: Beer) {
        if(pattern.matcher(beer.beerName).find()) {
            println("SQL Injection for beer name")
            return
        }
        jdbcTemplate.update("INSERT INTO beers(name, price) VALUES (?, ?)",
            beer.beerName, beer.beerPrice)
    }
}
```



```

    override fun getBeers(): String {
        val result: MutableList<Beer?> = jdbcTemplate.query("SELECT * FROM
beers", BeerRowMapper())
        var stringResult: String = ""
        for (item in result) {
            stringResult += item
        }
        return stringResult
    }

    override fun getBeerByName(name: String): String? {
        if(pattern.matcher(name).find()) {
            println("SQL Injection for beer name")
            return null
        }
        val result: Beer? = jdbcTemplate.queryForObject("SELECT * FROM beers
WHERE name = '$name'", BeerRowMapper())
        return result.toString()
    }

    override fun getBeerByPrice(price: Float): String {
        val result: MutableList<Beer?> = jdbcTemplate.query("SELECT * FROM beers
WHERE price <= $price", BeerRowMapper())
        var stringResult: String = ""
        for (item in result) {
            stringResult += item
        }
        return stringResult
    }

    override fun updateBeer(beer: Beer) {
        if(pattern.matcher(beer.beerName).find()) {
            println("SQL Injection for beer name")
            return
        }
        jdbcTemplate.update("UPDATE beers SET name = ?, price = ? WHERE id = ?",
beer.beerName, beer.beerPrice, beer.beerID)
    }

    override fun deleteBeer(name: String) {
        if(pattern.matcher(name).find()) {
            println("SQL Injection for beer name")
            return
        }
        jdbcTemplate.update("DELETE FROM beers WHERE name = ?", name)
    }
}

```

SQLiteExample

```
package com.sd.laborator

import org.springframework.boot.autoconfigure.EnableAutoConfiguration
import org.springframework.boot.autoconfigure.SpringBootApplication
import org.springframework.boot.runApplication
import org.springframework.context.annotation.ComponentScan

@SpringBootApplication
open class SQLiteExample

fun main(args: Array<String>) {
    runApplication<SQLiteExample>(*args)
}
```

Application.properties

```
spring.datasource.url=jdbc:sqlite:beer.db
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=student
spring.rabbitmq.password=student
sqliteexample.rabbitmq.queue=sqliteexample.queue1
sqliteexample.rabbitmq.exchange=sqliteexample.direct
sqliteexample.rabbitmq.routingkey=sqliteexample.routingkey
```

ATENTIE AICI! PROBABIL VA TREBUI SA ITI FACI TU PE LOC CONTUL DE RABBIT MQ.
VEZI IN LAB

Pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/maven-v4_0_0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.sd.laborator</groupId>
    <artifactId>SQLiteExample</artifactId>
    <version>1.0.0</version>
    <packaging>jar</packaging>
```

```
<name>com.sd.laborator SQLiteExample</name>

<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>2.1.9.RELEASE</version>
  <relativePath/> <!-- lookup parent from repository -->
</parent>

<properties>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <kotlin.version>1.3.70</kotlin.version>
  <kotlin.code.style>official</kotlin.code.style>
  <junit.version>4.12</junit.version>
</properties>

<repositories>
  <repository>
    <id>jcenter</id>
    <url>https://jcenter.bintray.com/</url>
  </repository>
</repositories>

<dependencies>
  <dependency>
    <groupId>khttp</groupId>
    <artifactId>khttp</artifactId>
    <version>1.0.0</version>
  </dependency>
  <dependency>
    <groupId>org.jetbrains.kotlin</groupId>
    <artifactId>kotlin-stdlib</artifactId>
    <version>${kotlin.version}</version>
  </dependency>
  <dependency>
    <groupId>org.jetbrains.kotlin</groupId>
    <artifactId>kotlin-test-junit</artifactId>
    <version>${kotlin.version}</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>${junit.version}</version>
    <scope>test</scope>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
```

```

        <artifactId>spring-boot-starter-jdbc</artifactId>
    </dependency>
    <dependency>
        <groupId>org.xerial</groupId>
        <artifactId>sqlite-jdbc</artifactId>
        <version>3.28.0</version>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-amqp</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-web</artifactId>
        <version>5.1.10.RELEASE</version>
    </dependency>
</dependencies>

<build>
    <sourceDirectory>src/main/kotlin</sourceDirectory>
    <testSourceDirectory>src/test/kotlin</testSourceDirectory>

    <plugins>
        <plugin>
            <groupId>org.jetbrains.kotlin</groupId>
            <artifactId>kotlin-maven-plugin</artifactId>
            <version>${kotlin.version}</version>
            <executions>
                <execution>
                    <id>compile</id>
                    <phase>compile</phase>
                    <goals>
                        <goal>compile</goal>
                    </goals>
                    <configuration>
                        <compilerPlugins>
                            <plugin>spring</plugin>
                        </compilerPlugins>
                    </configuration>
                </execution>
                <execution>
                    <id>test-compile</id>
                    <phase>test-compile</phase>
                    <goals>
                        <goal>test-compile</goal>
                    </goals>
                </execution>
            </executions>
        </plugin>
    </plugins>
</build>
<dependencies>

```

```
        <dependency>
            <groupId>org.jetbrains.kotlin</groupId>
            <artifactId>kotlin-maven-allopen</artifactId>
            <version>${kotlin.version}</version>
        </dependency>
    </dependencies>
</plugin>

    <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
</plugins>
</build>

</project>
```